joining between them in native SQL. On top of Trino we can then place a standard BI tool like Tableau, Looker, or Metabase to also make the data available in the form of dashboards and the like.

As in the preceding section, this architecture is a more specific tool than the first two explained in this chapter. So instead of discussing the workflows and data mesh principles, we'll focus on why this architecture is more of a special-purpose tool.

8.4.3 Considerations

This architecture is powerful if your company is focused on building data-intensive applications, especially in real time, as the native integrations provided by Kafka make building and combining these easy.

This architecture is designed to be used a lot by development teams. To accommodate data engineers and data scientists, we need to add a few extras, like Trino. We also do not have any native integration into a notebook-like environment as the Databricks-based architecture had. We can make all of this work; it just requires extra attention.

Another consideration is the schema registry and the data catalog. Kafka will need its own schema registry to be fully useful. The schema registry then needs to be integrated into an external data catalog. This is also the case for all other architectures, but in all other architectures, the data catalog could also serve as a schema registry; in this case, we do need the Kafka-specific registry.

Most variations used in other architectures are possible here as well. In particular, installing more data engineering–heavy components is the one that will make the most sense here.

8.4.4 Kafka architecture summary

The Kafka architecture is different from the other three we have covered. While the focus of the other three is to enable analytical work, this architecture focuses more on building data-heavy applications, which do not necessarily need to be analytical in nature.